

Estruturas de Dados — LISTA ARRAY + LISTA DUPLAMENTE ENCADEADA

LISTA ARRAY

TestesLista.java

```
package Implementação.Lista.Array;

public class TestesLista {
    public static void main(String[] args) throws ListaVazia {

        ListaArray check = new ListaArray(5);

        check.insertLast("A");
        check.insertLast("B");
        check.insertLast("C");

        System.out.println(check.first());
        System.out.println(check.last());
        System.out.println(check.size());

        check.insertFirst("Z");

        System.out.println(check.before(2));
        System.out.println(check.after(1));

        check.replaceElement(1, "X");

        check.swapElements(0, 1);

        check.remove(0);

        System.out.println(check.size());
    }
}
```

Lista.java

```
package Implementação.Lista.Array;

public interface Lista {

    boolean isFirst(int i);
    boolean isLast(int i);

    Object first() throws ListaVazia;
    Object last() throws ListaVazia;

    Object before(int i) throws ListaVazia;
    Object after(int i) throws ListaVazia;

    Object replaceElement(int i, Object o) throws ListaVazia;

    void swapElements(int i, int j) throws ListaVazia;

    void insertBefore(int i, Object o);
    void insertAfter(int i, Object o);

    void insertFirst(Object o);
    void insertLast(Object o);

    Object remove(int i) throws ListaVazia;

    int size();
    boolean isEmpty();
    void grow();
}
```

ListaArray.java

```
package Implementação.Lista.Array;

public class ListaArray implements Lista {

    private Object[] arr;
    private int capacidade, size, inicio;

    public ListaArray(int capacidade){

        this.capacidade = capacidade;
        this.size = 0;
    }
}
```

```

        this.inicio = 0;

        arr = new Object[capacidade];
    }

    public int size(){
        return size;
    }

    public boolean isEmpty(){
        return size == 0;
    }

    public boolean isFirst(int i){
        return i == inicio;
    }

    public boolean isLast(int i){
        return i == size() - 1;
    }

    public Object first() throws ListaVazia{
        if (isEmpty()) throw new ListaVazia("Lista Vazia!");
        return arr[inicio];
    }

    public Object last() throws ListaVazia{
        if (isEmpty()) throw new ListaVazia("Lista Vazia!");
        return arr[size() - 1];
    }

    public Object before(int i) throws ListaVazia{
        if (i == inicio) throw new ListaVazia("Sem anterior!");
        return arr[i - 1];
    }

    public Object after(int i) throws ListaVazia{
        if (i == size() - 1) throw new ListaVazia("Sem próximo!");
        return arr[i + 1];
    }

    public Object replaceElement(int i, Object o) throws ListaVazia{

        if (isEmpty()) throw new ListaVazia("Lista Vazia!");

        Object antigo = arr[i];

```

```

        arr[i] = o;

        return antigo;
    }

    public void swapElements(int i, int j) throws ListaVazia{

        if (isEmpty()) throw new ListaVazia("Lista Vazia!");

        Object temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }

    public void insertFirst(Object o){

        for (int i = size(); i > inicio; i--){
            arr[i] = arr[i - 1];
        }

        arr[inicio] = o;
        size++;
    }

    public void insertLast(Object o){

        arr[size()] = o;
        size++;
    }

    public void insertBefore(int i, Object o){

        for (int r = size(); r > i; r--){
            arr[r] = arr[r - 1];
        }

        arr[i] = o;
        size++;
    }

    public void insertAfter(int i, Object o){

        for (int r = size(); r > i + 1; r--){
            arr[r] = arr[r - 1];
        }
    }

```

```

        arr[i + 1] = o;
        size++;
    }

    public Object remove(int i) throws ListaVazia{

        if (isEmpty()) throw new ListaVazia("Lista Vazia!");

        Object remover = arr[i];

        for (int r = i; r < size() - 1; r++){
            arr[r] = arr[r + 1];
        }

        size--;

        return remover;
    }

    public void grow(){

        int nova = capacidade * 2;
        Object[] novo = new Object[nova];

        for (int i = 0; i < size(); i++){
            novo[i] = arr[i];
        }

        arr = novo;
        capacidade = nova;
    }
}

```

ListaVazia.java

```

package Implementação.Lista.Array;

public class ListaVazia extends RuntimeException {

    public ListaVazia(String err){
        super(err);
    }
}

```

Testes Lista Array

```
package Implementação.Lista.Array;

public class TestesLista {
    public static void main(String[] args) throws ListaVazia {
        ListaArray check = new ListaArray(5);

        check.insertLast("A");
        check.insertLast("B");
        check.insertLast("C");

        System.out.println(check.first());
        System.out.println(check.last());
        System.out.println(check.size());

        check.insertFirst("Z");

        System.out.println(check.before(2));
        System.out.println(check.after(1));

        check.replaceElement(1, "X");

        check.swapElements(0, 1);

        check.remove(0);

        System.out.println(check.size());
    }
}
```

LISTA DUPLAMENTE ENCADEADA

No.java

```
package Implementação.Lista.Duplamente;

public class No {

    private Object elemento;
    private No proximo, anterior;
```

```

    public void setAnterior(No anterior){
        this.anterior = anterior;
    }

    public void setProximo(No proximo){
        this.proximo = proximo;
    }

    public void setElement(Object elemento){
        this.elemento = elemento;
    }

    public Object getElement(){
        return elemento;
    }

    public No getAnterior(){
        return anterior;
    }

    public No getProximo(){
        return proximo;
    }
}

```

Lista.java

```

package Implementação.Lista.Duplamente;

public interface Lista {

    boolean isFirst(int i);
    boolean isLast(int i);

    Object first() throws ListaVazia;
    Object last() throws ListaVazia;

    Object before(int i) throws ListaVazia;
    Object after(int i) throws ListaVazia;

    Object replaceElement(int i, Object o) throws ListaVazia;

    void swapElements(int i, int j) throws ListaVazia;
}

```

```

void insertBefore(int i, Object o);
void insertAfter(int i, Object o);

void insertFirst(Object o);
void insertLast(Object o);

Object remove(int i) throws ListaVazia;

int size();
boolean isEmpty();
}

```

ListaDuplamenteEncadeada.java

```

package Implementação.Lista.Duplamente;

public class ListaDuplamenteEncadeada implements Lista {

    private int size;
    private No head, tail;

    public ListaDuplamenteEncadeada(){

        this.head = new No();
        this.tail = new No();

        head.setProximo(tail);
        tail.setAnterior(head);

        size = 0;
    }

    public int size(){
        return size;
    }

    public boolean isEmpty(){
        return size == 0;
    }

    public boolean isFirst(int i){
        return i == 0;
    }
}

```



```

public boolean isLast(int i){
    return i == size() - 1;
}

public Object first() throws ListaVazia{
    if (isEmpty()) throw new ListaVazia("Lista Vazia");
    return head.getProximo().getElement();
}

public Object last() throws ListaVazia{
    if (isEmpty()) throw new ListaVazia("Lista Vazia");
    return tail.getAnterior().getElement();
}

public Object before(int i) throws ListaVazia{

    No atual = head.getProximo();

    for (int j = 0; j < i; j++){
        atual = atual.getProximo();
    }

    return atual.getAnterior().getElement();
}

public Object after(int i) throws ListaVazia{

    No atual = head.getProximo();

    for (int j = 0; j < i; j++){
        atual = atual.getProximo();
    }

    return atual.getProximo().getElement();
}

public Object replaceElement(int i, Object o) throws ListaVazia{

    No atual = head.getProximo();

    for (int j = 0; j < i; j++){
        atual = atual.getProximo();
    }

    Object antigo = atual.getElement();
    atual.setElement(o);
}

```

```

        return antigo;
    }

    public void swapElements(int i, int j) throws ListaVazia{

        No a = head.getProximo();
        No b = head.getProximo();

        for (int k = 0; k < i; k++) a = a.getProximo();
        for (int k = 0; k < j; k++) b = b.getProximo();

        Object temp = a.getElement();
        a.setElement(b.getElement());
        b.setElement(temp);
    }

    public void insertBefore(int i, Object o){

        No atual = head.getProximo();

        for (int j = 0; j < i; j++){
            atual = atual.getProximo();
        }

        No anterior = atual.getAnterior();
        No novo = new No();

        novo.setElement(o);

        novo.setAnterior(anterior);
        novo.setProximo(atual);

        anterior.setProximo(novo);
        atual.setAnterior(novo);

        size++;
    }

    public void insertAfter(int i, Object o){

        No atual = head.getProximo();

        for (int j = 0; j < i; j++){
            atual = atual.getProximo();
        }
    }

```

```

        No proximo = atual.getProximo();
        No novo = new No();

        novo.setElement(o);

        novo.setAnterior(atual);
        novo.setProximo(proximo);

        atual.setProximo(novo);
        proximo.setAnterior(novo);

        size++;
    }

    public void insertFirst(Object o){

        No atual = head.getProximo();
        No novo = new No();

        novo.setElement(o);

        novo.setAnterior(head);
        novo.setProximo(atual);

        atual.setAnterior(novo);
        head.setProximo(novo);

        size++;
    }

    public void insertLast(Object o){

        No atual = tail.getAnterior();
        No novo = new No();

        novo.setElement(o);

        novo.setAnterior(atual);
        novo.setProximo(tail);

        atual.setProximo(novo);
        tail.setAnterior(novo);

        size++;
    }

```

```

    public Object remove(int i) throws ListaVazia{

        No atual = head.getProximo();

        for (int j = 0; j < i; j++){
            atual = atual.getProximo();
        }

        No ant = atual.getAnterior();
        No prox = atual.getProximo();

        ant.setProximo(prox);
        prox.setAnterior(ant);

        size--;

        return atual.getElement();
    }
}

```

ListaVazia.java

```

package Implementação.Lista.Duplamente;

public class ListaVazia extends RuntimeException {

    public ListaVazia(String err){
        super(err);
    }
}

```

Testes Lista Dupla

```

package Implementação.Lista.Duplamente;

public class TestesLista {

    public static void main(String[] args) {

        ListaDuplamenteEncadeada lista = new ListaDuplamenteEncadeada();

        lista.insertLast("A");
    }
}

```

```
        lista.insertLast("B");
        lista.insertLast("C");

        System.out.println(lista.first());
        System.out.println(lista.last());
        System.out.println(lista.size());

        lista.insertFirst("0");

        lista.insertBefore(2, "X");

        System.out.println(lista.replaceElement(2, "Z"));

        lista.swapElements(0, 2);

        System.out.println(lista.remove(1));

        System.out.println(lista.size());
    }
}
```